

438. Find All Anagrams in a String Medium

Acceptance: 53.1% | [LeetCode](#)

Given two strings `s` and `p`, return an array of all the start indices of `p`'s anagrams in `s`. You may return the answer in **any order**.

Example 1:

```
**Input:** s = "cbaebabacd", p = "abc"
**Output:** [0,6]
**Explanation:**
The substring with start index = 0 is "cba", which is
an anagram of "abc".
The substring with start index = 6 is "bac", which is
an anagram of "abc".
```

Example 2:

```
**Input:** s = "abab", p = "ab"
**Output:** [0,1,2]
**Explanation:**
The substring with start index = 0 is "ab", which is
an anagram of "ab".
The substring with start index = 1 is "ba", which is
an anagram of "ab".
The substring with start index = 2 is "ab", which is
an anagram of "ab".
```

Constraints:

- `1 <= s.length, p.length <= 3 * 104`
- `s` and `p` consist of lowercase English letters.

Claude Opus Solution

Find All Anagrams in a String

1. Intuition

An **anagram** of `p` is any rearrangement of `p`'s characters. The crucial insight is that **two strings are anagrams if and only if they have identical character counts** — order doesn't matter.

So the problem reduces to: *find all windows of length `len(p)` in `s` whose character frequency matches `p`'s character frequency.*

This is a perfect fit for the **sliding window** pattern because: - The window size is **fixed** (always `len(p)`). - As we slide one step, only **two characters change** (one enters on the right, one leaves on the left), so we can update counts in $O(1)$ instead of recounting.

2. Approach (Pseudocode)

- If `len(p) > len(s)`: return `[]` (no anagram possible)
- Build frequency count of `p` → `p_count`
- Build frequency count of first window of size `len(p)` in `s` → `window_count`
- `result = []`
- If `window_count == p_count`: append index `0`
- Slide the window from left to right:
for `i` from `len(p)` to `len(s)-1`:
 - Add `s[i]` to window (right side enters)
 - Remove `s[i - len(p)]` from window (left side leaves)
 - If `window_count == p_count`: append start index `(i - len(p) + 1)`
- return `result`

3. Python Solution

```
from typing import List
from collections import Counter

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        n, k = len(s), len(p)

        # Edge case: p is longer than s, no anagram can exist.
        if k > n:
```

```

    return []

    # p_count: target frequency map we want every valid window to match.
    p_count = Counter(p)

    # window_count: frequency map of the CURRENT window in s.
    # Initialize it with the first window (the first k characters).
    window_count = Counter(s[:k])

    result = []

    # Check the very first window before we start sliding.
    if window_count == p_count:
        result.append(0)

    # Slide the window one character at a time.
    # 'i' is the index of the NEW character entering on the right.
    for i in range(k, n):
        # --- Add the new character on the right ---
        window_count[s[i]] += 1

        # --- Remove the old character on the left ---
        # The char leaving is at index (i - k), since the window has size k.
        left_char = s[i - k]
        window_count[left_char] -= 1
        # Clean up zero entries so Counter equality works correctly.
        # (Counters with explicit 0 counts are NOT equal to ones without them.)
        if window_count[left_char] == 0:
            del window_count[left_char]

        # --- Compare current window with target ---
        # The window now spans indices [i - k + 1, i].
        if window_count == p_count:
            result.append(i - k + 1)

    return result

```

Why the `del` matters

`Counter({'a': 1, 'b': 0}) != Counter({'a': 1})`. If we don't delete keys that drop to zero, the equality check `window_count == p_count` can fail incorrectly. Deleting keeps the comparison clean.

4. Complexity Analysis

Time Complexity: $O(n)$ - Building `p_count` and the first window: $O(k)$. - The sliding loop runs $n - k$ times. Each iteration does $O(1)$ work for the increment/decrement, **but** the `window_count == p_count` comparison touches at most 26 keys (lowercase English letters). Since 26 is a constant, each comparison is $O(1)$. - Total: **$O(n)$** where $n = \text{len}(s)$.

Space Complexity: $O(1)$ - Both counters hold at most 26 distinct lowercase letters $\rightarrow$ bounded by a constant. - The output list isn't counted as auxiliary space.

5. Alternative Approaches

A) Fixed-size array + match counter (fastest constant factor)

Instead of comparing two `Counter` objects each step (which, though $O(1)$, has overhead), track a single integer `matches` representing how many of the 26 letters currently have the exact right count. Update `matches` incrementally as characters enter/leave the window, and report an index whenever `matches == 26`.

```

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        n, k = len(s), len(p)
        if k > n:
            return []

        need = [0] * 26
        win = [0] * 26
        for c in p:
            need[ord(c) - 97] += 1

```

```

# matches = number of letters where win[c] == need[c]
matches = sum(1 for i in range(26) if win[i] == need[i])

res = []
for i in range(n):
    ci = ord(s[i]) - 97
    # add char entering on the right
    if win[ci] == need[ci]: matches -= 1 # was matching, now will break
    win[ci] += 1
    if win[ci] == need[ci]: matches += 1 # now it matches

    # remove char leaving on the left, once window exceeds size k
    if i >= k:
        cj = ord(s[i - k]) - 97
        if win[cj] == need[cj]: matches -= 1
        win[cj] -= 1
        if win[cj] == need[cj]: matches += 1

    if matches == 26:
        res.append(i - k + 1)
return res

```

This avoids any dictionary/Counter overhead and only updates the `matches` counter in $O(1)$ per step.

B) Variable-size window (the reference solution)

The reference solution uses a clever variable-size window: it shrinks the left boundary whenever any count goes negative, and reports an index whenever the window naturally reaches size `len(p)`. It's elegant and also $O(n)$, but slightly less obvious to reason about than the fixed-window version.

C) Brute force (for understanding only)

For each starting index `i` in `s`, sort the substring `s[i:i+k]` and compare with sorted `p`. This is $O(n \cdot k \log k)$ — too slow for the constraints, but conceptually simple.

Recommended takeaway: This is a textbook **fixed-size sliding window with frequency counting** problem. Recognizing "anagram $\Rightarrow$ identical character counts" + "fixed window $\Rightarrow$ slide and update in $O(1)$ " is the key pattern to internalize.

Codex Solution

1. Intuition

Two strings are anagrams if they contain the **same characters with the same frequencies**.

So, for every substring of `s` with length `len(p)`, we want to know:

Does this substring have the same character frequency as `p`?

Instead of rebuilding the frequency count for every substring, we use a **sliding window** of size `len(p)`:

- Add the new character entering the window.
- Remove the old character leaving the window.
- Compare the current window frequency with `p`'s frequency.

Because `s` and `p` only contain lowercase English letters, we can use arrays of size 26 for frequency counts.

2. Approach

Let:

```

m = len(p)
n = len(s)

```

If `m > n`, no substring of `s` can be an anagram of `p`.

Pseudocode

```

if len(p) > len(s):
    return []

```

```

create need[26] initialized to 0
create window[26] initialized to 0

for each character c in p:
    need[c] += 1

answer = []

for right from 0 to len(s) - 1:
    add s[right] to window

    if window size is greater than len(p):
        remove s[right - len(p)] from window

    if window size equals len(p):
        if window == need:
            append starting index of window to answer

return answer

```

3. Python Solution

```

from typing import List

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        # Lengths of the two strings
        n = len(s)
        m = len(p)

        # If p is longer than s, no substring of s can match p
        if m > n:
            return []

        # Frequency array for characters in p
        # need[i] stores how many times character chr(i + ord('a')) appears in p
        need = [0] * 26

        # Frequency array for the current sliding window in s
        window = [0] * 26

        # Build frequency count for p
        for ch in p:
            index = ord(ch) - ord('a')
            need[index] += 1

        result = []

        # Slide a window over s
        for right in range(n):
            # Add the current character to the window
            current_char_index = ord(s[right]) - ord('a')
            window[current_char_index] += 1

            # If window size exceeds m, remove the leftmost character
            if right >= m:
                left_char_index = ord(s[right - m]) - ord('a')
                window[left_char_index] -= 1

            # Once the window size is exactly m, check if it matches p's frequency
            if right >= m - 1:
                # If frequencies match, the current window is an anagram of p
                if window == need:
                    start_index = right - m + 1
                    result.append(start_index)

        return result

```

4. Complexity Analysis

Let:

```
n = len(s)
m = len(p)
```

Time Complexity

$O(n)$

We iterate through `s` once.

At each step, we compare two arrays of size `26`, which is constant because the alphabet is fixed to lowercase English letters.

So the total time is:

$O(26 * n) = O(n)$

Space Complexity

$O(1)$

We use two arrays of size `26`.

Since `26` is constant, the space usage is:

$O(1)$

5. Alternative Approaches

1. Hash Map / Counter Sliding Window

Instead of arrays, use `collections.Counter` to store character frequencies.

```
Counter(window) == Counter(p)
```

This is clean but usually slower than fixed-size arrays.

2. Sort Every Substring

For every substring of length `len(p)`, sort it and compare it with sorted `p`.

This is much slower.

Time complexity:

$O(n * m \log m)$

3. Optimized Sliding Window with Match Count

Maintain how many of the `26` character counts currently match between the window and `p`.

This avoids comparing the full array every time, but the implementation is slightly more complex.

The time complexity is still:

$O(n)$

The fixed-size frequency array solution is usually the best balance of simplicity and efficiency.